



A TALK FOR PEOPLE WHO BUILD FOR THE WEB

Rethinking the web layout.

Why your next project should use a compiled language: one syntax for text, boxes, media, state and style — and a compiler that does the heavy lifting before the page ever reaches a browser.

A terminal window with a dark background and light grey text. The title bar shows three window control buttons and the word 'terminal'. The terminal contains the following text:

```
$ wf init hello --template static
Created hello/ with AGENTS.md
$ wf build
3 pages, 0 errors, 0 warnings
$ wf build -d report
PDF: 9 page(s)
$ wf build -d deck
Slides: 16 slide(s)
```

Below the terminal window are five buttons: 'one syntax', 'one binary', 'web', 'PDF', and 'slides'. The 'web' button is white with dark grey text, while the others are light blue with dark blue text.

Six files before the first box.

```
a new project, today

$ ls -a my-site
package.json package-lock.json
tsconfig.json vite.config.ts
postcss.config.js tailwind.config.js
eslint.config.js .prettierrc
node_modules/ ... hundreds of packages
$ ls src/
six configs in, not one line of layout
```

01 Configuration before content

A bundler, a CSS pipeline, a type checker and a linter each want a file before anything is laid out.

02 Dependencies nobody chose

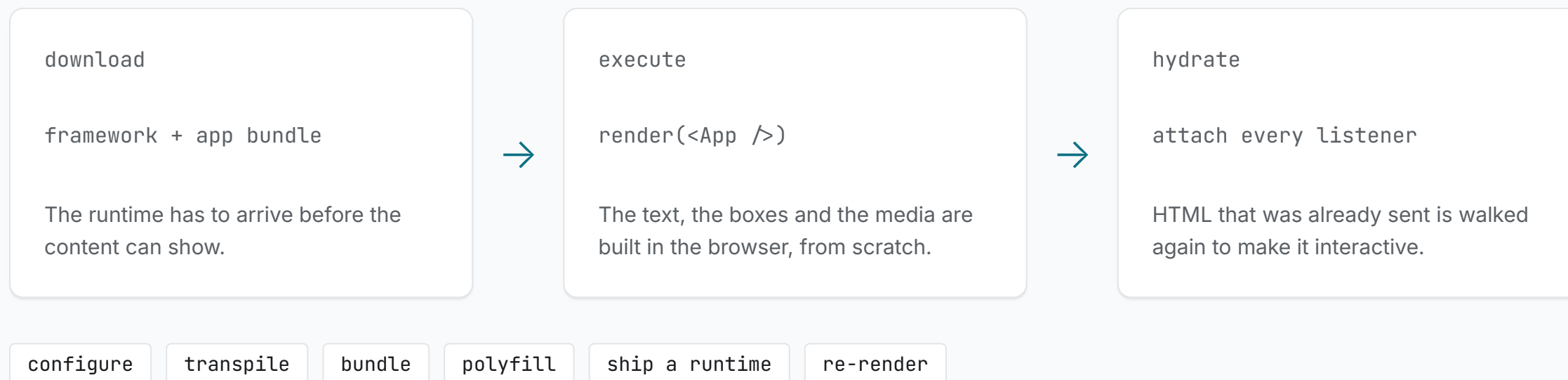
One install brings a tree of packages to audit, update and keep from breaking.

03 Tools that cannot see each other

The type checker cannot read the CSS, the linter cannot read the routes — so the browser finds the mistakes.

And the reader's browser finishes the job.

A typical stack ships a framework with the page, and the layout is assembled on the reader's device — on every visit, on every phone.



01

A new method

Describe the layout in one language a compiler understands, and do the heavy lifting once, at build time — not on every reader's device.

One syntax for layout, state and behaviour.

WHAT YOU WRITE

Home.wf

```
src/pages/Home.wf

page Home(path: "/", title: "Home") {
  state count = 0
  derived doubled = count * 2

  Container {
    Heading("Welcome").h1
    Text("Count: {count} · doubled {doubled}")
    Button("+1").primary {
      on click { count = count + 1 }
    }
  }
}
```

WHAT THE READER GETS

index.html · app.js



localhost:3000/

Welcome

Count: 3 · doubled 6

3

+1

No config file, no package to install, no virtual DOM:
the one text node that reads count repaints when it
changes.

Seven shapes. That is the whole grammar.

Text, boxes and media are written the same way as state and style. The compiler, the linters and the editor read one registry of what each element takes.

WRITTEN	WHAT IT IS
<code>page · component · store</code>	A declaration — a lowercase keyword names what is being made
<code>Row · Card · Image</code>	An element — text, boxes and media, capitalised, from one registry
<code>.primary .lg .h1</code>	A flag — a boolean prop, or one case of an enum prop
<code>gap: .md</code>	A prop — passed by name; one positional may lead
<code>Card { ... }</code>	A block — the element's children: the layout is the nesting
<code>on click { ... }</code>	A handler — what happens when the reader acts
<code>style { padding: \$md }</code>	Raw CSS — with design tokens and live {values}

The heavy lifting happens before anyone visits.

Pre-render	Every page written as HTML, so the first paint needs no script	Prune	One stylesheet: only the components and tokens the site uses
Cut	Of 37 runtime modules, only those the program reaches ship	Resize	Every image at each width, with its size, so nothing shifts
Check	Types, props, routes, translations and accessibility — every build	Secure	A Content-Security-Policy the build holds its own output to
Index	Sitemap, canonical links, Open Graph and JSON-LD for every page	Compress	A .gz beside every text file, ready for the host to serve

MEASURED

And the compile step is fast.

wf is one native binary. Wall-clock time of wf build on the projects in the repository's examples and tests.

17 ms a three-page marketing site, start to finish	19 ms a three-page dashboard with a store and a keyed table	0.21 s a nine-page PDF report: charts, tables, fonts embedded	0.16 s this deck: sixteen slides, five pictures, two QR codes
1 binary: compiler, server, formatter, tests	0 node_modules, bundler configs or plugins	3.7 kB gzipped app.js for a page of static text	0 headless browsers launched to make a PDF

A mistake stops at the compiler, not in production.

```
error[T05]: `User` has no field `nmae`  
  → src/pages/Profile.wf:3:18  
  |  
3 |   Heading(user.nmae).h1  
  |               ^^^^  
  = help: Its fields are `id`, `name`  
  = fix:  nmae → name
```

116 codes, one registry

The build, the editor, wf explain and the guide read the same list.

fix A fix where one is known

The nearest name, a ?., the missing props — applied by the editor.

1× Every mistake, once

Every file is read even when another does not parse.

02

Beyond the screen

The same layout compiles to a PDF document and a slide deck — laid out by the compiler itself. No headless browser to spin up.

One component. Three surfaces.

```
src/Talk.wf

component Point(_ title: String, body: String) {
  Card { Heading(title).h3 Text(body) }
}

page Site(path: "/", title: "Talk") {
  Container { for p in POINTS { Point(p.title, body: p.body) } }
}

page Handout(path: "/handout", title: "Handout") {
  Document(size: "A4") {
    Footer { Text("Page {page} of {pages}") }
    for p in POINTS { Point(p.title, body: p.body) }
  }
}

page Talk(path: "/talk", title: "Talk") {
  Presentation {
    for p in POINTS { Slide { Point(p.title, body: p.body) } }
  }
}
```

web

On screen

wf build: the HTML, the CSS, and a runtime cut to what the page reaches.

pdf

On paper

-f pdf: A4 pages with a numbered footer, each Card in its own CSS.

deck

On stage

-f slides: a 16:9 slide for every point, from the same loop.

Pick the output in one line of config.



Static site

Every page pre-rendered, then hydrated. Sitemap, canonical, Open Graph.

```
"ssg": true
```



Single-page app

A page per chunk, loaded when its route shows; a service worker on request.

```
"spa"
```



PDF

The page's own CSS, laid out on paper: running headers, contents, charts.

```
"pdf"
```



Slide deck

A slide to a page, from components and data — like this one.

```
"slides"
```



Custom elements

Components as tags for React, Vue, Rails or a plain page.

```
"elements"
```

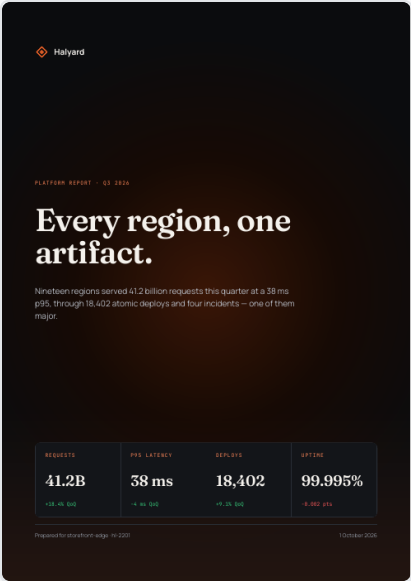
webfluent.app.json

```
{ "build": { "output_type": "slides",  
             "slides": { "size": "16:9" } } }
```

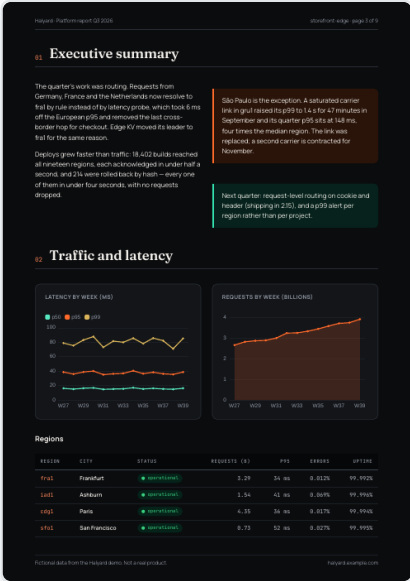
The program does not change. A PDF is refused what only a browser can do — a button, a handler, a route — where it is written, and the editor offers only what the output draws.

Paper, without a headless browser.

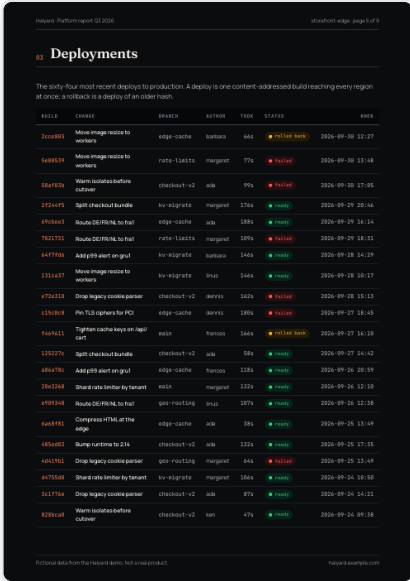
The compiler's own paged engine lays out the page's HTML and CSS — the cascade, flex and grid, shaped Arabic, tables that repeat their header. Nothing to install, nothing to launch.



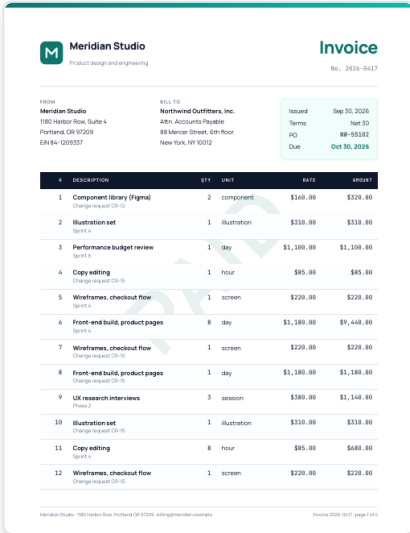
Report cover — a radial glow and stat tiles



Charts side by side, a table by content



64 rows, the header on every page



An invoice: band, watermark, totals kept whole



Arabic, right to left, from the same engine

You are looking at a .wf file.

```
examples/decks/webfluent/src/Deck.wf

component Opener(_ title: String, num: String, line: String) {
  Slide(class: "section") {
    Text(num, class: "section__num")
    Text(title, class: "section__title")
    Text(line, class: "section__line")
  }
}

page Deck(path: "/", title: "Rethinking the web layout") {
  Presentation {
    Opener("A new method", num: "01",
      line: "Write it once. Compile it.")
    Slide {
      Head("Seven shapes.", over: "One unified syntax")
      for g in GRAMMAR { Row { Text(g.form) Markdown(g.means) } }
    }
  }
}
```

Every slide here is a component — an opener, a heading, a console frame — written once and placed sixteen times. A for makes a row, or a slide, from each item of a list.

The fonts are files beside it, the colours the docs site's tokens; wf build writes the PDF, and nothing else runs.



THE SOURCE OF EVERY DECK

THE CHALLENGE

Try it on one page you already have.

Rebuild it in WebFluent, then measure the difference yourself — every number on these slides is one command away.

RUN	WHAT YOU WILL SEE
<code>wf init try-it -t static</code>	Start — one command, one folder, nothing to install after the binary
<code>src/pages/Home.wf</code>	Rebuild one page you have — layout, state and style in a single file
<code>wf serve</code>	Watch it — a save rebuilds and reloads; a mistake is drawn over the page
<code>wf check</code>	Count what it caught — every finding with a code, a place and a fix
<code>wf build --stats</code>	Weigh it — every file gzipped, and each runtime module that shipped
<code>wf verify</code>	Time it — every route loaded in a real browser: errors, paint, bytes

YOUR MOVE

Spin up a layout.

See how much cleaner it gets.

```
$ curl -sSL .../install.sh | bash
```

```
$ wf init my-site --template static
```

```
$ wf serve
```



webfluent.monzeromer.dev